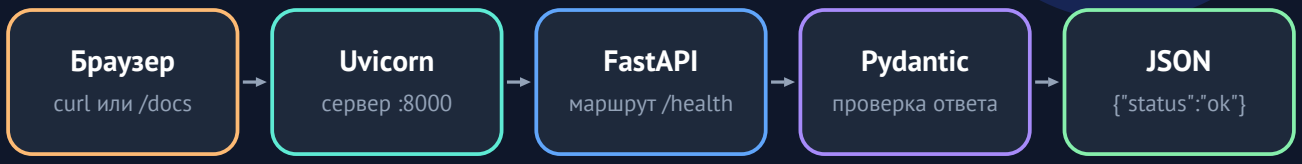
УЧЕБНЫЙ ПРОЕКТ № 2 · ВЫМЫШЛЕННАЯ КОМПАНИЯ INDUSTRIALFLOW

AI Data Analyst

Этап 01 · Подготовка проекта

Наглядное руководство: окружение uv, первый API на FastAPI, настройки, тесты, проверки качества и CI. Всё, на что лягут база данных, аналитика и AI-агент.



Числа в будущих отчётах считает код. Сегодня — фундамент для этого кода.

1 / 10	7 / 7	9 / 9	38	1
этап курса завершён	автотестов прошли	локальных проверок PASS	пакетов зафиксировано в uv.lock	маршрут API: GET /health

Что внутри

1 Этап за минуту
2 Путь запроса
3 Окружение и uv
4 Создание проекта
5 Импорт и sys.path
6 HTTP и JSON
7 Разбор кода API
8 Настройки
9 Тесты
10 Ruff и mypy
11 CI на GitHub Actions
12 Git и GitHub
13 Шпаргалка команд
14 Типичные ошибки
15 Проверь себя
16 Итоги проверки

Проверка 2026-10-20 · коммит 4448e55 · uv 0.12.23 · Python 3.12.15 · macOS, Apple Silicon

Подробный текст этапа — docs/STAGE_01.md; журнал и отчёт — docs/DEVELOPMENT_LOG.md, docs/VERIFICATION.md

1 Этап за минуту

Что уже работает, чего ещё нет и где этот этап в плане курса.

На первом этапе создан каркас будущего ИИ-аналитика компании **IndustrialFlow**. Приложение пока умеет одно: отвечать на `GET /health`. Зато вокруг него уже есть всё, что понадобится дальше: управление зависимостями, настройки, тесты, проверки качества и CI. Следующие этапы будут добавлять функциональность в этот каркас, не переделывая его.

API

FastAPI-приложение с маршрутом GET /health и Pydantic-моделью ответа. Документация /docs строится сама.

Настройки

Класс Settings читает параметры из переменных окружения и .env. Пароли и ключи не попадут в код.

Тесты

7 тестов pytest: HTTP-ответы, OpenAPI, приоритет настроек, совпадение версий. Все проходят.

Качество

Ruff проверяет стиль, ошибки и импорты и форматирует код. mypy в строгом режиме проверяет типы.

CI

GitHub Actions повторяет все проверки на чистой Linux-машине при каждом push и pull request.

Воспроизводимость

uv.lock фиксирует версии 38 пакетов; берутся только выпуски старше двух недель.

Где мы в плане курса

1. готово — **Подготовка проекта**
2. далее — **PostgreSQL и схема БД**
3. Генерация данных
4. Аналитическое ядро
5. Поиск аномалий
6. REST API
7. AI-агент
8. Интерфейс и отчёты
9. Тесты и безопасность
10. Docker и публикация

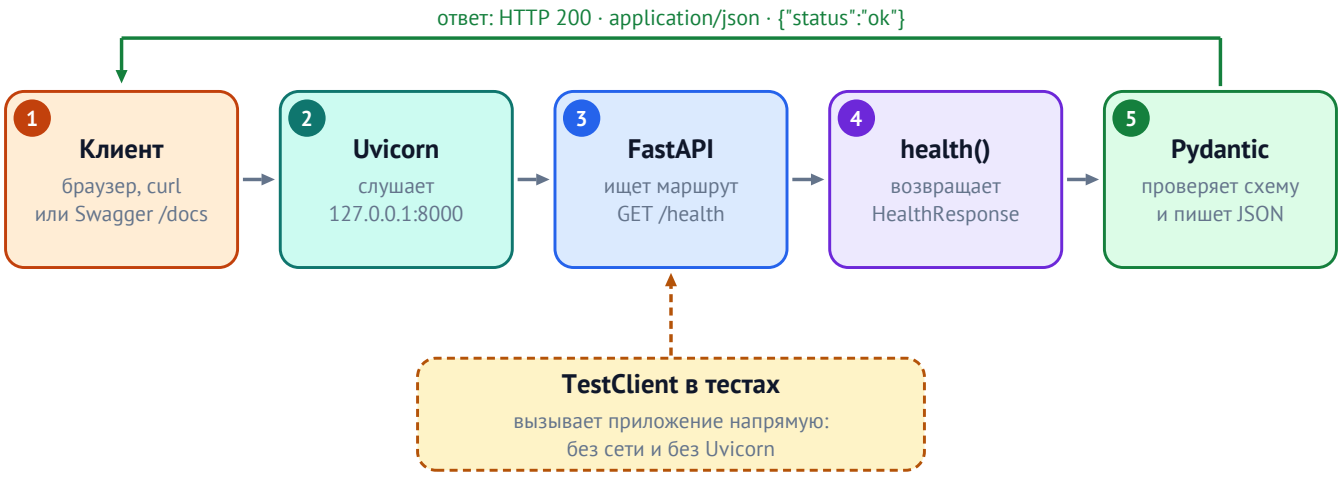
Чего ещё нет: базы данных, демонстрационных данных, аналитики и LLM — это этапы 02–07. Сейчас важно понять цепочку «запрос → сервер → приложение → ответ» и инструменты, которые будут проверять каждый следующий шаг.

Главный принцип проекта

Числа считает код, а не LLM. Выручку, средний чек, сравнения периодов и аномалии всегда рассчитывает детерминированный SQL или Python. Модель понимает вопрос, выбирает инструменты и объясняет результат. Поэтому с первого этапа так важны тесты и проверки: каждое число в будущем отчёте должно быть подтверждено кодом, который проверен автоматически.

2 Путь запроса

Что происходит между нажатием Execute в /docs и ответом {"status": "ok"}.



Шаг	Кто	Что делает
1	Клиент	Формирует HTTP-запрос GET /health и отправляет его на 127.0.0.1:8000.
2	Uvicorn	Принимает соединение, разбирает HTTP и передаёт запрос приложению по протоколу ASGI.
3	FastAPI	Ищет обработчик по паре «метод + путь». Для POST ответит 405, для неизвестного пути — 404.
4	health()	Создаёт объект HealthResponse(status="ok").
5	Pydantic	Проверяет объект по схеме ответа и превращает его в JSON.

Фактический ответ сервера

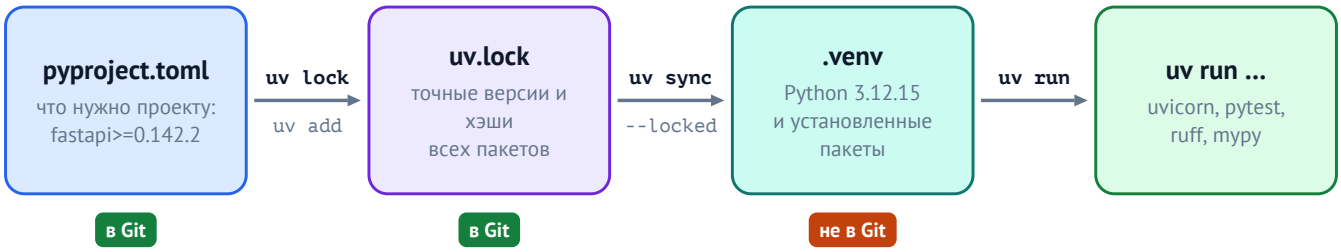
```
$ curl -i http://127.0.0.1:8000/health
HTTP/1.1 200 OK
server: uvicorn
content-length: 15
content-type: application/json

{"status":"ok"}
```

TestClient в тестах подключается сразу к FastAPI (шаг 3): без сети, без Uvicorn и без свободного порта. Поэтому все тесты проходят за 0.27 с.

3 Окружение и uv

Как uv превращает список требований в одинаковое окружение на любом компьютере.



Виртуальное окружение `.venv` — отдельная папка со своим Python и библиотеками только для этого проекта. Так проекты не мешают друг другу. Системный `python3` на этом Mac — 3.9.6; его не трогаем, всё запускаем через `uv run`.

	pyproject.toml	uv.lock
Кто пишет	Человек и команда uv add	Только uv
Что внутри	Требования: fastapi>=0.142.2	Точные версии и хэши файлов
Транзитивные пакеты	Нет	Да, все 38: starlette, pydantic, anyio…
В Git	Да	Да

Зависимости проекта (версии из uv.lock)

Пакет	Версия	Группа	Зачем
`fastapi`	`0.142.2`	runtime	маршруты, проверка данных, /docs
`pydantic-settings`	`2.15.0`	runtime	настройки из окружения и .env
`uvicorn`	`0.54.0`	runtime	ASGI-сервер, слушает порт
`httpx2`	`2.13.1`	dev	HTTP-клиент для TestClient
`mypy`	`2.4.0`	dev	проверка типов
`pytest`	`9.1.1`	dev	запуск тестов
`ruff`	`0.16.10`	dev	линтер и форматировщик
`starlette`	`1.7.0`	через fastapi	веб-основа FastAPI
`pydantic`	`2.13.5`	через fastapi	модели и проверка данных

Политика возраста пакетов. `[tool.uv] exclude-newer = "2026-10-06T00:00:00Z"` — uv не видит выпуски PyPI новее этой даты. Свежий выпуск бывает сломан или даже вредоносен; за две недели такие проблемы обычно находят. Самый новый пакет в lock — websockets 17.2 от 3 октября.

4 Создание проекта

Какими командами создан проект, чем они отличаются от стартового задания и что лежит в каждой папке.

```
# uv 0.12.23 — та же версия, что закреплена в CI
curl -LsSf https://astral.sh/uv/0.12.23/install.sh \
  | env UV_NO_MODIFY_PATH=1 sh

uv init --app --no-package --python 3.12 --author-from none
rm main.py && mkdir -p app tests
uv add fastapi 'uvicorn[standard]' pydantic-settings
uv add --dev pytest ruff mypy httpx2
```

uv init в uv 0.12

Без флагов создаёт пакет-библиотеку: src/ai_data_analyst/, [build-system] и [project.scripts]. Для веб-сервиса с папкой app/ в корне нужен --app --no-package.

httpx2 вместо httpx

Starlette 1.7 предупреждает: «Using httpx with starlette.testclient is deprecated; install httpx2 instead». TestClient теперь работает на httpx2.

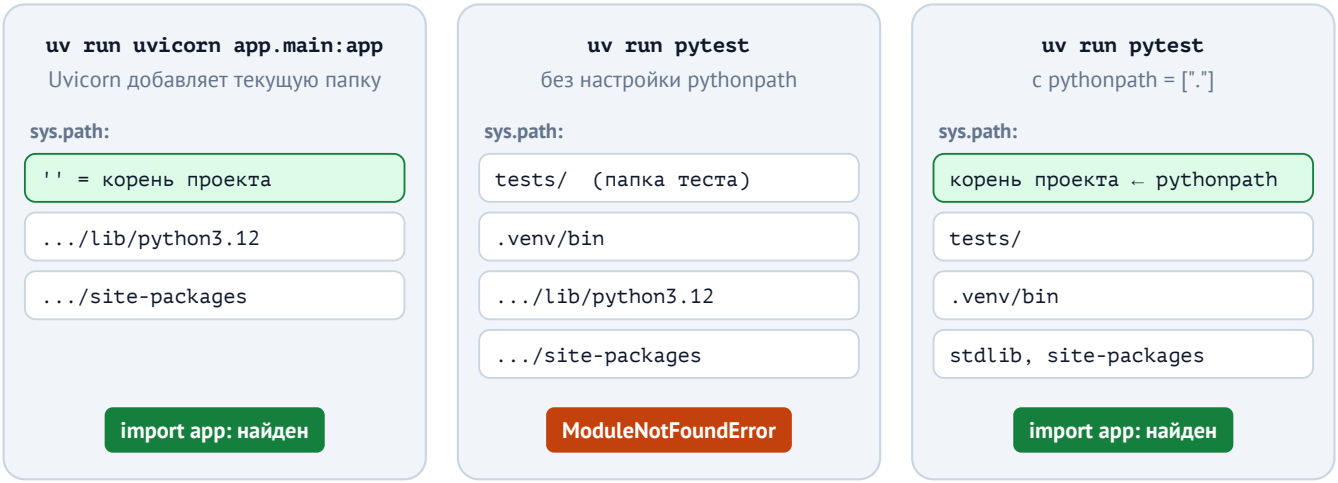
Карта файлов

Файл или папка	Для чего нужен
app/__init__.py	Делает app пакетом; хранит версию __version__ = "0.1.0".
app/config.py	Класс Settings и готовый объект settings.
app/main.py	Объект FastAPI и маршрут GET /health.
tests/	Три файла тестов: API, настройки, версия.
pyproject.toml	Описание проекта, зависимости и настройки pytest, Ruff, mypy, uv.
uv.lock	Точные версии всех 38 пакетов.
.python-version	3.12 — какой Python выбирает uv.
.env.example	Образец локальных настроек; сам .env — не в Git.
.gitignore	Что Git не отслеживает: .venv, кеши, .env.
.github/workflows/ci.yml	Проверки в GitHub Actions.
docs/	Руководство этапа, журнал, отчёт, ADR, этот PDF.
scripts/build_guide.py	Сборка этого PDF.
AGENTS.md	Правила работы над проектом.

5 Импорт и sys.path

Почему сервер находит пакет app, а pytest без настройки — нет.

Модуль — один файл `.py`. **Пакет** — папка с модулями; файл `__init__.py` выполняется при первом импорте пакета. **sys.path** — список папок, где Python ищет модули. Строка `from app.main import app` работает, только если в одной из этих папок лежит папка app.



Фактическая ошибка до настройки

```
$ uv run pytest
ImportError while importing test module '.../tests/test_health.py'.
E   ModuleNotFoundError: No module named 'app'
Interrupted: 3 errors during collection
```

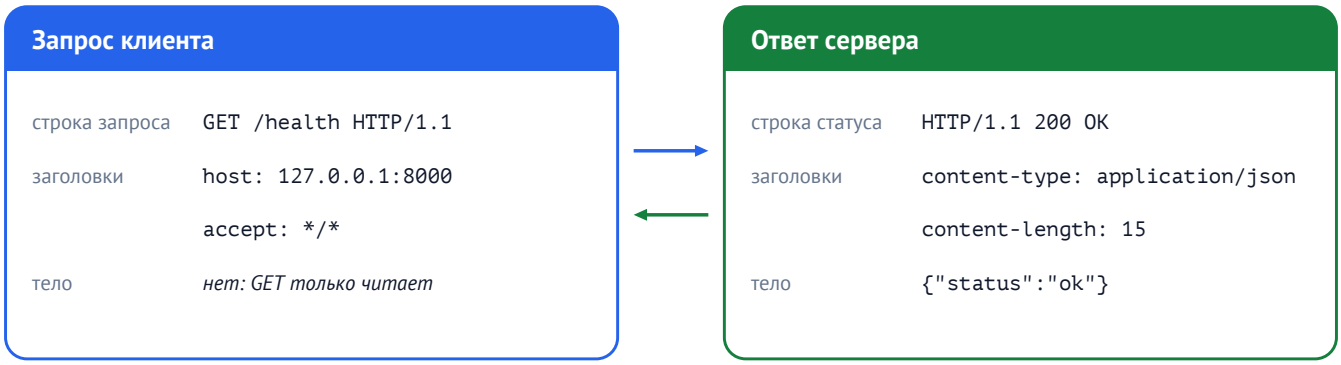
Исправление в pyproject.toml

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

Почему так. Проект создан с --no-package: в .venv ставятся только зависимости, сам app не устанавливается. Другие решения — запускать python -m pytest (флаг -m добавляет текущую папку) или оформить проект пакетом. Явная настройка работает при любом способе запуска.

6 HTTP и JSON

Из чего состоят запрос и ответ и какие коды встретятся в проекте.



Метод	Смысл	В этом проекте
`GET`	Получить данные, ничего не меняя	GET /health; позже GET /api/metrics/revenue
`POST`	Создать объект или запустить действие	позже — вопрос AI-агенту
`PUT` / `PATCH`	Заменить или частично изменить	пока не используем
`DELETE`	Удалить	пока не используем

Код	Значение	Когда увидишь (проверено)
200 OK	Успех	GET /health → 200 OK
404 Not Found	Такого пути нет	GET /nope → 404 Not Found
405 Method Not Allowed	Путь есть, метод не тот	POST /health → 405 Method Not Allowed, заголовок allow: GET
422 Unprocessable Content	Неверные входные данные	появится на этапе 06 с параметрами запросов
500 Internal Server Error	Ошибка на сервере	эксперимент в разделе 7

JSON — текстовый формат данных: объекты {...}, массивы [...], строки, числа, true / false / null. FastAPI превращает Pydantic-модель в JSON сам, а `response.json()` в тесте превращает JSON обратно в словарь Python.

7 Разбор кода API

app/main.py построчно и эксперимент с неправильным ответом.

```
"""Точка входа API: создаём приложение FastAPI и служебный маршрут /health."""

from typing import Literal

from fastapi import FastAPI
from pydantic import BaseModel

from app import __version__
from app.config import settings

# Этот объект находит Uvicorn по адресу app.main:app, его же вызывают тесты.
# Название и версия попадают в OpenAPI-схему и на страницу /docs.
app = FastAPI(
    title=settings.app_name,
    description="AI-powered business analytics platform for IndustrialFlow",
    version=__version__,
)


class HealthResponse(BaseModel):
    """Ответ проверки живости: процесс запущен и отвечает на HTTP-запросы."""

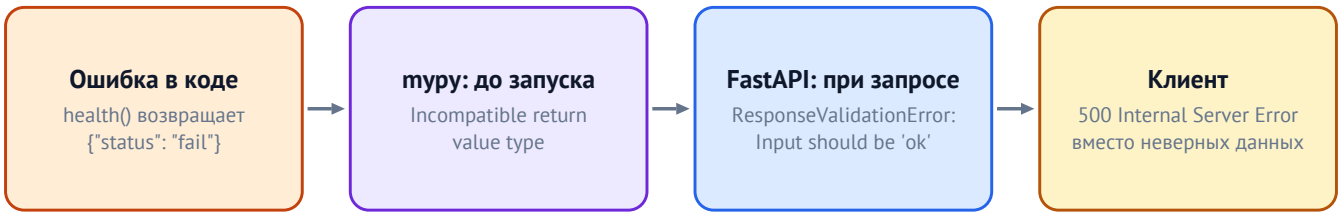
    # Literal допускает ровно одно значение: ответ с другим FastAPI не отправит.
    status: Literal["ok"]


@app.get("/health")
def health() -> HealthResponse:
    """Сообщить, что API работает; базу данных этот маршрут не проверяет."""
    return HealthResponse(status="ok")
```

Фрагмент	Что делает
`import ...`	Три группы: стандартная библиотека, сторонние пакеты, наш код. Порядок проверяет правило Ruff I.
`app = FastAPI(...)`	Объект приложения. Его находит Uvicorn (app.main:app) и вызывает TestClient. title и version — в /docs.
`class HealthResponse`	Pydantic-модель: описание формы ответа. Literal["ok"] разрешает ровно одно значение.
`@app.get("/health")`	Регистрирует функцию для пары GET + /health. Другие методы на этом пути получат 405.
`-> HealthResponse`	Аннотация возврата становится схемой ответа: по ней строится /docs и проверяется результат функции.
`def, не async def`	Обычную функцию FastAPI выполняет в отдельном потоке. К async вернёмся на этапе 06.

Эксперимент: неправильный ответ

Во временной копии проекта функция health() вернула `{"status": "fail"}`. Фактические результаты:



Два уровня защиты: типы ловят ошибку в коде, Pydantic — в данных. Неправильный ответ не уходит клиенту молча. Для аналитики это главное: лучше явная ошибка, чем неверное число в отчёте.

Модель → JSON → словарь

FastAPI превращает объект модели в JSON-текст, а тест превращает JSON обратно в словарь Python. Значения ниже получены из настоящего приложения при сборке PDF:

```
>>> HealthResponse(status="ok").model_dump_json()   # модель → JSON
'{"status":"ok"}'
>>> client.get("/health").json()                    # JSON → dict
{'status': 'ok'}
```

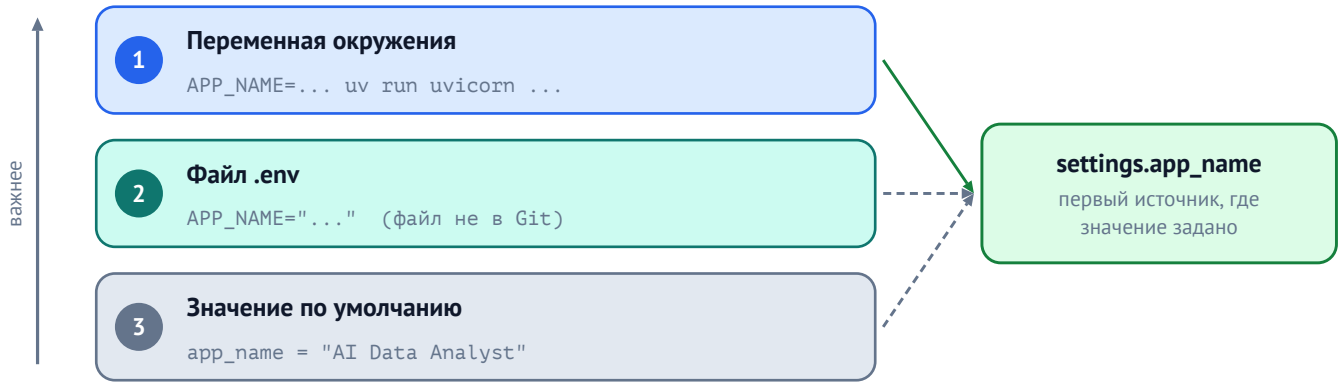
OpenAPI и страница /docs

FastAPI собирает описание всех маршрутов и моделей в документ OpenAPI (`GET /openapi.json`). Страница /docs — это Swagger UI, который рисует этот документ. Схема HealthResponse из приложения; docstring модели стал полем description:

```
{
  "properties": {
    "status": {
      "type": "string",
      "const": "ok",
      "title": "Status"
    }
  },
  "type": "object",
  "required": [
    "status"
  ],
  "title": "HealthResponse",
  "description": "Ответ проверки живости: процесс запущен и отвечает на HTTP-запросы."
}
```

8 Настройки

Конфигурация живёт в окружении, а не в коде.




```
"""Настройки приложения из переменных окружения и локального файла .env."""

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    """Конфигурация, которая читается при создании объекта настроек.

    Приоритет источников: переменная окружения → строка в .env → значение
    по умолчанию ниже. Регистр имён не важен: поле app_name читается из
    переменной APP_NAME.
    """

    # extra="ignore": позже в .env появятся ключи Docker и PostgreSQL,
    # которые к этим настройкам не относятся; без него Pydantic отклонил бы их.
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

    # Название сервиса в OpenAPI-схеме и на странице /docs.
    app_name: str = "AI Data Analyst"


# Один экземпляр на процесс: модули импортируют уже прочитанные настройки.
settings = Settings()
```

Тест	Что закрепляет
`test_default_app_name`	Без окружения и .env берётся значение из кода.
`test_env_file_overrides_default`	Строка в .env заменяет значение по умолчанию.
`test_environment_variable_beats_env_file`	Переменная окружения важнее строки в .env.

extra="ignore" — позже в .env появятся переменные Docker и PostgreSQL, не относящиеся к Settings. **settings = Settings()** создаётся один раз при импорте, поэтому изменения .env видны только после перезапуска сервера: --reload следит за .py-файлами.

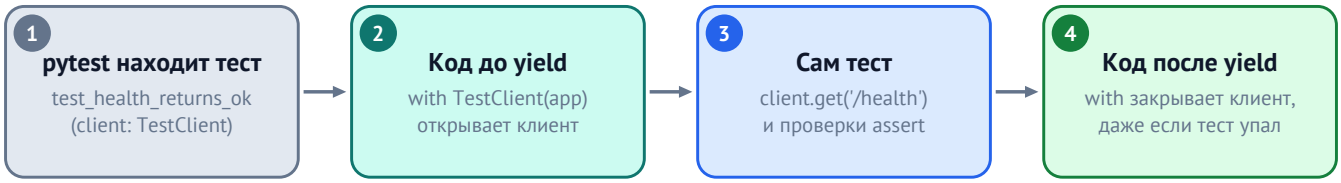
```
APP_NAME="IndustrialFlow Analytics" uv run uvicorn app.main:app
```

Заголовок страницы /docs сменится на «IndustrialFlow Analytics».

9 Тесты

Как pytest находит тесты, что такое фикстура и что доказывает каждый тест.

Pytest находит файлы `test_*.py` в папке tests/ и выполняет функции с именем `test_...`. Внутри обычный assert: если условие ложно, pytest покажет, какие значения не совпали. Фикстура — подготовка для теста; тест получает её, объявив параметр с тем же именем.




```
@pytest.fixture
def client() -> Iterator[TestClient]:
    """Открыть тестовый клиент: запросы идут прямо в приложение, без сети.

    Фикстура выполняется перед каждым тестом, который принимает параметр
    client, а код после yield закрывает клиент после теста.
    """
    with TestClient(app) as test_client:
        yield test_client
```

Тест	Что доказывает
`test_health_returns_ok`	GET /health → 200 и ровно {"status": "ok"}
`test_health_rejects_post`	POST /health → 405
`test_openapi_describes_health`	Схема знает версию, путь и обязательное поле status
`test_default_app_name`	Умолчание из кода
`test_env_file_overrides_default`	.env важнее умолчания
`test_environment_variable_beats_env_file`	Окружение важнее .env
`test_version_matches_pyproject`	app.__version__ совпадает с pyproject.toml

Встроенные фикстуры: **monkeypatch** временно меняет переменные окружения и всё возвращает; **tmp_path** даёт новую пустую папку. Каждый тест — «подготовка → действие → проверка», части разделены пустыми строками.

10 Ruff и mypy

Два инструмента, которые находят ошибки до запуска программы.

Правило Ruff	Что ищет
E	Нарушения стиля PEP 8
F	Ошибки: неиспользуемые импорты, неопределённые имена
I	Порядок импортов
UP	Устаревший синтаксис, который в Python 3.12 проще
B	Типичные ловушки, например изменяемое значение аргумента по умолчанию

`ruff check .` ищет проблемы, с `--fix` исправляет безопасные; `ruff format .` переформатирует код, а `ruff format --check .` только проверяет — так в CI. Все команды собраны в шпаргалке (раздел 13).

mypy strict требует аннотации у каждой функции и не пропускает незаметные Any. Плагин `pydantic.mypy` объясняет mypy, какие аргументы принимают модели и настройки. Пример пользы — эксперимент из раздела 7: mypy нашёл неверный тип ответа до запуска.

> **Ruff 0.16 форматирует и Python-блоки внутри Markdown.** Примеры кода в docs/STAGE_01.md тоже проходят ruff format --check, поэтому документация не расходится со стилем кода.

Пример: что находит Ruff

Файл с типичными недочётами проверен с настройками проекта. Сначала код, затем фактический вывод:

```
from app.config import settings
import os
from typing import List


def names(items: List[str], seen=[]) -> List[str]:
    return [item for item in items if item not in seen]
```

```
$ uv run ruff check --output-format concise example.py
example.py:1:1: I001 [*] Import block is un-sorted or un-formatted
example.py:1:24: F401 [*] `app.config.settings` imported but unused
example.py:2:8: F401 [*] `os` imported but unused
example.py:3:1: UP035 `typing.List` is deprecated, use `list` instead
example.py:6:18: UP006 [*] Use `list` instead of `List` for type annotation
example.py:6:34: B006 Do not use mutable data structures for argument defaults
example.py:6:41: UP006 [*] Use `list` instead of `List` for type annotation
Found 7 errors.
[*] 5 fixable with the `--fix` option
    (1 hidden fix can be enabled with the `--unsafe-fixes` option).
```

I001 — порядок импортов, F401 — неиспользуемый импорт, UP035 и UP006 — List вместо list, B006 — общий список по умолчанию, который будет копить значения между вызовами. Пометка [*] — Ruff исправит сам через --fix.

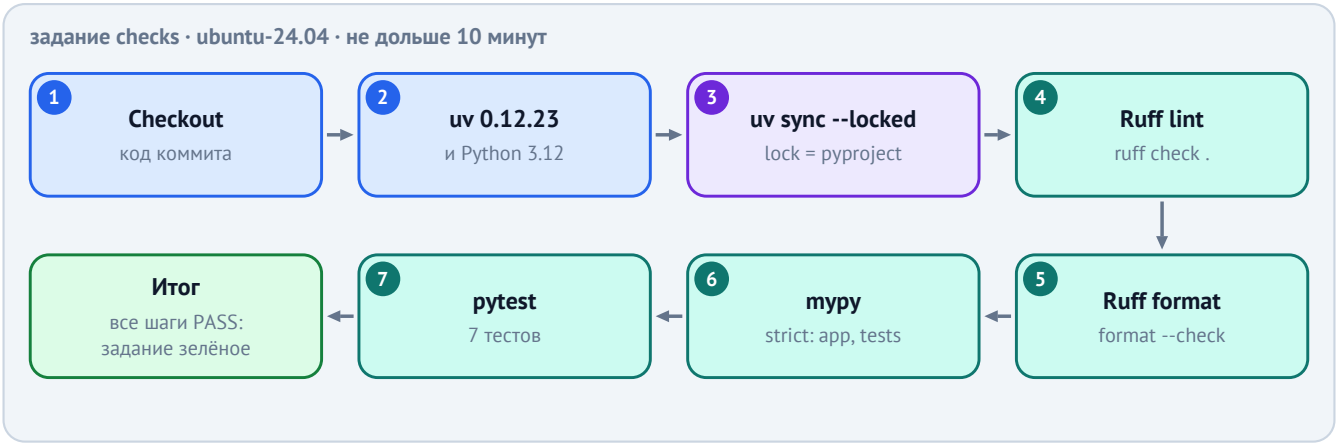
Пример: что находит mypy

```
$ uv run mypy app
app/main.py:30: error: Incompatible return value type
    (got "dict[str, str]", expected "HealthResponse")  [return-value]
Found 1 error in 1 file (checked 3 source files)
```

11 CI на GitHub Actions

Те же проверки на чистой Linux-машине при каждом изменении.

Запуск: push в main | pull request в main | вручную: workflow_dispatch



Настройка	Зачем
`permissions: contents: read`	Заданию разрешено только читать код.
`concurrency`	Новый push в тот же PR отменяет устаревшую проверку.
`uv sync --locked`	Ошибка, если uv.lock расходится с pyproject.toml.
`checkout@3d3c42e… # v7.0.1`	Действия закреплены полным SHA: тег автор может перенести на другой код, SHA — нет.
`persist-credentials: false`	Токен GitHub не остаётся в рабочей папке после checkout.

Статус удалённого CI: не проверен. Репозиторий на GitHub ещё не создан, поэтому удалённый запуск Actions не выполнялся. Workflow проверен actionlint, его команды прошли в чистом клоне.

Шаги проверок в ci.yml

```
# --locked завершает проверку ошибкой, если uv.lock расходится с pyproject.
- name: Install locked dependencies
  run: uv sync --locked

- name: Ruff lint
  run: uv run --locked ruff check .

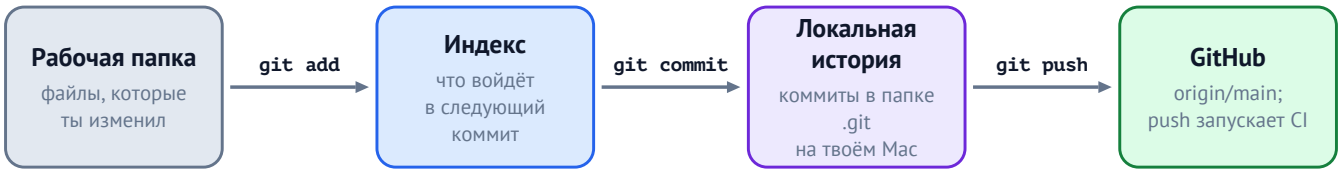
- name: Ruff format
  run: uv run --locked ruff format --check .

- name: Mypy
  run: uv run --locked mypy app tests

- name: Pytest
  run: uv run --locked pytest
```

12 Git и GitHub

Что хранится в репозитории и как опубликовать его на GitHub.



В Git

Код app/ и tests/, pyproject.toml, uv.lock, .python-version, .env.example, .gitignore, workflow, документация и этот PDF.

Не в Git

.venv (пересоздаётся uv sync), кеши инструментов, .env с личными настройками и будущими секретами.

Коммиты этапа

`4448e55` — код, тесты, настройки и CI; следующий коммит — руководство, журнал, отчёт и PDF. История: `git log --oneline`.

Публикация

1. На github.com: **New repository**, имя ai-data-analyst, **без** README, .gitignore и лицензии — иначе история на GitHub разойдётся с локальной.

2. Подключить удалённый репозиторий и отправить ветку:

```
git remote add origin https://github.com/<твой-логин>/ai-data-analyst.git
git push -u origin main
```

3. Открыть вкладку **Actions**: задание «Lint, types and tests» должно стать зелёным. Только после этого CI подтверждён.

> Сообщения коммитов — на английском, в повелительном наклонении: «Add …», «Fix …». Каждый коммит — законченный шаг, после которого проходят все проверки.

13 Шпаргалка команд

Все команды этапа на одной странице.

Задача	Команда
Создать окружение по lock	`uv sync --locked`
Запустить сервер	`uv run uvicorn app.main:app --reload`
Сервер на другом порту	`uv run uvicorn app.main:app --port 8001`
Проверить ответ	`curl -i http://127.0.0.1:8000/health`
Документация API	`http://127.0.0.1:8000/docs`
Все тесты	`uv run pytest`
Подробно, по одному	`uv run pytest -v`
Только тесты настроек	`uv run pytest -k config`
Стоп на первой ошибке	`uv run pytest -x`
Линтер	`uv run ruff check .`
Форматирование	`uv run ruff format .`
Типы	`uv run mypy app tests`
Добавить библиотеку	`uv add <пакет>`
Добавить инструмент	`uv add --dev <пакет>`
Проверить lock	`uv lock --check`
Прямые зависимости	`uv tree --depth 1`
Кто занял порт 8000	`lsof -i :8000`
История коммитов	`git log --oneline`
Пересобрать PDF	`uv run --with 'reportlab==4.4.9' python scripts/build_guide.py`

14 Типичные ошибки

Симптом, причина и что сделать.

Симптом	Причина	Что сделать
`uv: command not found`	Терминал открыт до установки uv	Открыть новую вкладку Terminal
`ModuleNotFoundError: No module named 'app'`	Нет pythonpath или команда не из корня	Проверить pyproject.toml, выполнить cd в корень проекта
`Address already in use`	Порт 8000 занят	`lsof -i :8000` или `--port 8001`
`405 Method Not Allowed`	Не тот HTTP-метод	Для /health — GET
`404 Not Found`	Опечатка в пути	Сверить путь с /docs
`uv sync --locked` завершается ошибкой	pyproject.toml изменён, lock — нет	`uv lock` и закоммитить uv.lock
Изменения в `.env` не видны	Настройки читаются при старте	Перезапустить сервер
`python3 --version` → 3.9.6	Это системный Python	Запускать через `uv run`
Предупреждение про `httpx` в тестах	Starlette 1.7 ждёт httpx2	Уже исправлено в группе dev
`ruff format --check` ругается на .md	Python-блок в Markdown не отформатирован	`uv run ruff format .`

15 Проверь себя

Вопросы для самопроверки и самостоятельное задание.

1. Чем pyproject.toml отличается от uv.lock? Почему коммитим оба, а .venv — нет?
Ответ: В pyproject.toml — требования к прямым зависимостям, в uv.lock — точные версии и хэши всех 38 пакетов. .venv пересоздаётся из lock и зависит от платформы.

2. Почему Uvicorn находит app, а pytest без настройки — нет?
Ответ: Uvicorn добавляет текущую папку в sys.path; pytest добавляет tests/, а проект не установлен как пакет. pythonpath = ["."] добавляет корень.

3. Что будет, если health() вернёт {"status": "fail"}?
Ответ: mypy найдёт ошибку типа до запуска; при запросе FastAPI выбросит ResponseValidationError, клиент получит 500.

4. Чем Uvicorn отличается от FastAPI?
Ответ: FastAPI — приложение: маршруты, проверка данных, документация. Uvicorn — сервер: слушает порт и передаёт запросы приложению по ASGI.

5. В каком порядке Settings выбирает app_name?
Ответ: Переменная окружения → строка в .env → значение по умолчанию.

6. Почему действия в CI закреплены SHA, а не тегом v7?
Ответ: Тег можно переписать на другой код, SHA коммита — нет.

Самостоятельное задание

> Добавь маршрут **GET /version**. Ответ описан Pydantic-моделью VersionResponse: name — из settings.app_name, version — из app.__version__. Напиши тест на код 200 и тело ответа. Готово, когда pytest, ruff check, ruff format --check и mypy проходят.

```
{"name": "AI Data Analyst", "version": "0.1.0"}
```

Подсказка: маршрут устроен так же, как /health.

16 Итоги проверки

Что фактически проверено 2026-10-20 на коммите 4448e55.

Окружение: macOS, Apple Silicon, uv 0.12.23, Python 3.12.15. Тесты: 7 прошли, 0 упали, 0 пропущены.

Проверка	Команда	Результат	Статус
Тесты	`uv run pytest`	7 passed, 0 failed, 0 skipped	**PASS**
Линтер	`uv run ruff check .`	All checks passed	**PASS**
Формат	`uv run ruff format --check .`	Все файлы отформатированы, включая Python-блоки в Markdown	**PASS**
Типы	`uv run mypy app tests`	Success: no issues found in 6 source files	**PASS**
Lock-файл	`uv lock --check`	uv.lock соответствует pyproject.toml	**PASS**
Чистый клон	`git clone → uv sync --locked → все проверки`	Коммит 4448e55 собирается и проходит проверки с нуля	**PASS**
Workflow	`actionlint .github/workflows/ci.yml`	actionlint 1.7.12: ошибок нет	**PASS**
Возраст пакетов	`даты upload-time в uv.lock`	38 пакетов; самый новый — websockets 17.2 от 2026-10-03	**PASS**
Сервер	`uv run uvicorn app.main:app + curl`	Uvicorn запустился; ответы ниже в таблице HTTP	**PASS**

Ответы сервера

Запрос	Код	Тело
`GET /health`	200 OK	`{"status":"ok"}`
`POST /health`	405 Method Not Allowed	`{"detail":"Method Not Allowed"}, allow: GET`
`GET /nope`	404 Not Found	`{"detail":"Not Found"}`
`GET /docs`	200 OK	`HTML-страница Swagger UI`

GitHub Actions: не проверен. Репозиторий на GitHub ещё не создан, поэтому удалённый запуск Actions не выполнялся. Workflow проверен actionlint, его команды прошли в чистом клоне.

Что дальше

- Создать репозиторий ai-data-analyst на GitHub, отправить ветку main и дождаться зелёного задания в Actions.
- Выполнить самостоятельное задание: маршрут GET /version с моделью VersionResponse и тестом.
- Этап 02: PostgreSQL в Docker Compose, модели SQLAlchemy 2 и первая миграция Alembic.